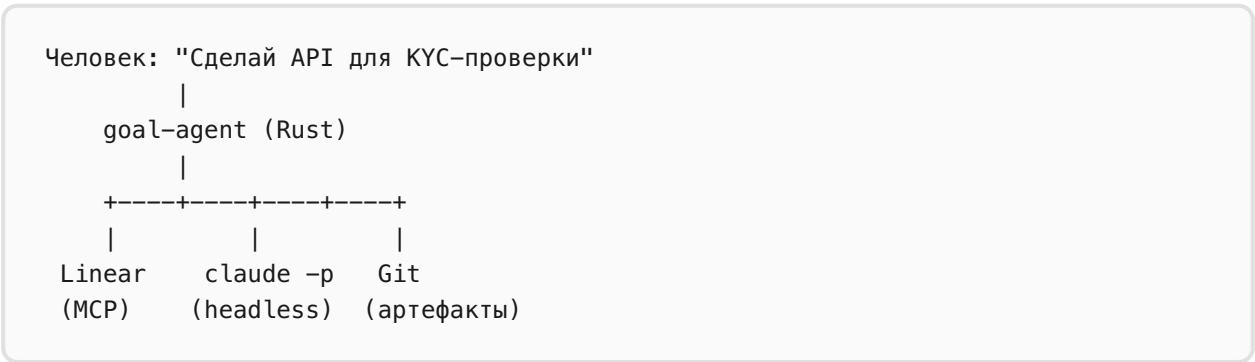


Goal Agent

Автономная целевая разработка · Архитектура v5 · Апрель 2026

1. Обзор

Rust CLI, который берёт задачу из Linear и доводит её до протестированного кода без участия человека. Человек взаимодействует с системой исключительно через Linear: отвечает на вопросы, разблокирует задачи, принимает решения.



Ключевые принципы

- **«Не могу» не принимается.** Замокай, заглуши, запиши в trade-offs, но продолжай работу.
- **Контексты обязательны.** Каждый агент читает project-context + task-context перед каждым действием.
- **Linear — единственный источник правды** для статусов, блокеров и коммуникации с человеком.
- **Все артефакты в git:** исследования, критерии приёмки, trade-offs, результаты тестов, история контекстов.

2. Роли

Роль	Когда	Что делает
Worker	Задача в Research / Develop / Testing	Исследует домен, кодит, тестирует. Читает все контексты перед каждым действием. Если не может что-то сделать — заявляет блокер, не пропускает.
Enabler	Worker заявляет «не могу X»	

		Расследует: правда не может? Проверяет CI, инфру, API, доки, конфиги. Либо «можешь, вот как», либо «подтверждённый blocker».
CEO Review	Задача в Review + Context Review	Жёсткий ревью качества: покрытие AC, фейковые тесты, false positives, ленивые контексты, пропущенные edge cases.
CEO Idle	Все задачи заблокированы / idle	Блокеры реальны? Что можно делать пока ждём? Можно ли разблокировать без человека?

Системные промпты

Worker: "Ты senior-разработчик. Читай контексты перед каждым действием. Кодь, тестируй. Если не можешь что-то сделать – НЕ пропускай, заяви blocker."

Enabler: "Ты DevOps/SRE. Тебе говорят 'не могу X'. Твоя задача – доказать что МОГУТ. Проверь CI, инфру, доки, конфиги. Если реально нельзя – обоснуй."

CEO Rev: "Ты жёсткий техдир. Ищи дыры: непокрытые AC, фейковые тесты, false positives прикрывающие лень, неполные контексты."

CEO Idle: "Проверь все блокеры. Реальны? Что можно делать пока ждём? Можно ли разблокировать без человека?"

3. Интеграция с Linear

Статусы задач

```
Todo --> Research --> Develop --> Testing --> Review --> Context Review --> Done
                        ^           ^           ^
                        +-----+-----+ (reject)
```

Любой статус --> Blocked (ожидание человека)
Blocked --> предыдущий статус (после резолюции)

Структура в Linear

```
Project (Linear Project)
+-- project-context      (описание проекта)
+-- false-positives      (CEO ревьют на предмет лени)
|
+-- Task (Linear Issue)
|   +-- task-context     (что пробовали, что не сработало)
|   +-- subtasks         (sub-issues = элементы работы)
|   +-- blockers         (blocked-by связи)
|   +-- comments         (дискуссии, резолюции, ревью CEO)
```

Теги коммуникации

Тег	Направление	Назначение
[QUESTION]	Агент → Человек	Нужно уточнение
[BLOCKER]	Агент → Человек	Нужен ресурс / доступ
[DECISION]	Агент → Человек	Нужен выбор из вариантов
[RESOLVED]	Человек → Агент	Ответил / предоставил ресурс
[DECIDED]	Человек → Агент	Принял решение
[WONTFIX]	Человек → Агент	Отменено человеком
[CONTEXT]	Агент	Уведомление об обновлении контекста
[CEO-REVIEW]	CEO	Результат ревью

4. Workflow

Фаза 1: Исследование (Research)

Worker:

1. Читает project-context.md + task-context.md
2. claude -p "Исследуй домен {goal}, найди решения на рынке, задокументируй"
3. Результат: research.md -> комментарий в Linear
4. Генерирует критерии приёмки -> ac.md
5. Разбивает на подзадачи -> sub-issues в Linear
6. Статус: Research -> Develop

Фаза 2: Разработка (Develop)

Worker (по каждой подзадаче):

1. Читает project-context + task-context
2. claude -p "Реализуй {subtask}, следуй AC"
3. Каждая подзадача: Todo -> Done
4. Если блокер -> проверка Enabler:
 - +-- Enabler: "можешь, вот как" -> Worker продолжает
| + task-context: "ложный блокер: {детали}"
 - +-- Enabler: "подтверждённый блокер" -> Linear [BLOCKER]
+ Blocked (ожидание человека)
5. Грабли записываются в task-context + project-context
6. Статус: Develop -> Testing

Фаза 3: Тестирование (Testing)

Worker:

1. claude -p "Протестируй всё по AC, найди непокрытые случаи"
2. Тесты падают -> обновить task-context, -> Develop
3. False positive -> записать в false-positives.md
4. Статус: Testing -> Review

Фаза 4: Ревью CEO

CEO Review:

1. claude -p "Жёсткий ревью: AC, тесты, edge cases, trade-offs"
2. Проверка: у каждого AC есть тест?
3. Проверка: каждый тест действительно проверяет то, что заявлено?
4. Проверка: false positives – не лень ли это?
5. Проверка: trade-offs обоснованы?
6. REJECT + конкретные претензии -> Develop
7. APPROVE -> Context Review

Фаза 5: Ревью контекстов (Дискуссия)

CEO читает BCE контексты разом:

- project-context.md
- false-positives.md
- все tasks/*/context.md

По каждой найденной проблеме – дискуссия:

CEO:	"Этот FP – реальный баг"	(претензия)
Автор:	"Только в parallel mode"	(защита)
CEO:	"Тогда запиши корректно"	(требование)
Автор:	"Обновил"	(принято)

-> RESOLVED

Правила:

- Максимум 4 раунда на issue, потом эскалация человеку
- Каждый раунд = конкретный аргумент, не повтор
- Результат = конкретное изменение в контексте
- История дискуссии сохраняется в комментариях Linear

Фаза 6: Взаимодействие с человеком

Агент постит [QUESTION] / [BLOCKER] / [DECISION]

Задача -> Blocked (ожидание человека)

|

Человек видит в Linear, отвечает:

[RESOLVED]	"API-ключ в .env.dev"
[DECIDED]	"Вариант Б, latency критична"
[WONTFIX]	"Эту проверку не делаем"

|

goal-agent poll loop обнаруживает резолюцию:

1. Читает резолюцию
2. Обновляет task-context: "Блокер X решён: {как}"
3. Обновляет project-context если полезно другим задачам
4. Enabler проверяет: ресурс доступен?
5. Разблокировка, Worker продолжает
6. Если были моки -> замена на реальную реализацию

5. Enabler — расследователь «не могу»

```
Worker: "Не могу задеплоить на dev"
|
Enabler: claude -p "Проверь возможно ли {действие}"
  Проверяет:
    +-- CI/CD:      glab ci, .gitlab-ci.yml, GitHub Actions
    +-- Docker:     compose-файлы, Dockerfile, запущенные контейнеры
    +-- Env:        .env.example, vault, 1password-cli
    +-- API:        sandbox-эндпоинты, тестовые ключи, recorded fixtures
    +-- Инфра:      Makefile, scripts/, docker-compose, README
    +-- Инструменты: доступные MCP-серверы, установленные CLI
    +-- Lint/Type:  eslint, tsc, cargo check, ruff
  |
  Результат A: "Можешь. Команда: {cmd}"
    -> Worker продолжает
    -> task-context: "Ложный блокер: Worker заявлял X, на деле Y"
  |
  Результат B: "Подтверждённый блокер. Нужно: {конкретный ресурс}"
    -> Linear [BLOCKER] с конкретным запросом
    -> project-context: "{ресурс} недоступен, причина"
```

6. Артефакты (git-tracked)

```
project/
+-- .goal-agent/
  +-- project-context.md      # Архитектура, стек, грабли
  +-- false-positives.md     # CEO ревьюит периодически
  +-- tasks/
    +-- {linear-task-id}/
      +-- context.md          # Что пробовали, что не сработало
      +-- research.md         # Результаты исследования
      +-- ac.md               # Критерии приёмки
      +-- trade-offs.md       # Что отсутствует и почему
```

7. Технологический стек

Компонент	Технология
Язык	Rust + tokio

Linear MCP	rmcp (официальный Rust MCP SDK v1.4) → SSE → mcp.linear.app
Claude	<code>claude -p headless c --allowedTools , --max-turns , --max-budget-usd</code>
Git	Артефакты в репозитории, PR на выходе
Коммуникация	Комментарии Linear (теги: QUESTION, BLOCKER, DECISION, RESOLVED)

8. Главный цикл (псевдокод)

```
loop {
  // 1. Новые задачи в Todo
  let todo = linear.search_issues(status: "Todo").await;
  if let Some(task) = todo.first() {
    start_research(task).await;
    continue;
  }

  // 2. Разблокированные задачи (человек ответил)
  let unblocked = find_newly_resolved_tasks().await;
  for task in unblocked {
    apply_resolution(task).await;
    resume_work(task).await;
  }

  // 3. Активные задачи
  let active = linear.search_issues(
    status: ["Research", "Develop", "Testing"]
  ).await;
  for task in active {
    match task.status {
      Research => do_research(task).await,
      Develop  => do_develop(task).await,
      Testing  => do_testing(task).await,
    }
  }

  // 4. Ревью завершённых задач
  let review = linear.search_issues(status: "Review").await;
  for task in review {
    ceo_review(task).await;
  }

  // 5. Ревью контекстов после одобрения
  let ctx_review = linear.search_issues(
    status: "Context Review"
  ).await;
  for task in ctx_review {
    ceo_context_review_with_discussion(task).await;
  }

  // 6. CEO Idle – когда всё заблокировано
  if all_tasks_blocked_or_done() {
    ceo_idle_check_blockers().await;
  }
}
```

```
    sleep(poll_interval).await;  
}
```

Проектное решение: один агент (Claude), четыре персоны (Worker, Enabler, CEO Review, CEO Idle). Разные системные промпты, одна модель. Это проще мульти-модельного подхода, а Claude на данный момент top-1 на всех релевантных бенчмарках.